

CSL351 : COMPUTER NETWORKS
Assignment - 8

Name - Ashish Ranjan

ID no. - 12340370

Folder Structure :

1. **as8_que1.cc** -> the main .cc code for the CCA i.e TCPAshish
2. **as8_gnuplot.p** -> the gnuplot code used for creating the 4 plots
3. **as8_run.sh** -> the shell script used for running the .cc code with different flow values and parameter for either using the TCPAshish or not
4. **as8_results folder** -> folder containing the .txt files from the different execution of .cc code via the shell script, the various plots output as [cwnd_plt,rtt_plt,ssthresh_plt,through_plt].png

Report from Next Page :

Setting up the custom CCA :

1.) Function used to set the CCA?

- I set the Congestion Control Algorithm (CCA) dynamically using the Config::Set function
- used Config::Set() to map my chosen TCP variants to the nodes in my topology. Inside a loop that iterates through my total flows, I determined whether a node should use my custom algorithm or the baseline by assigning a string ("ns3::TcpAshish" or "ns3::TcpNewReno").

2.) Line of code used to set the particular socket

```
Ipv4AddressHelper address;  
address.SetBase ("10.1.1.0", "255.255.255.0");  
address.Assign (routerDevices);  
  
std::vector<Ipv4InterfaceContainer> rightInterfaces;  
for (uint32_t i = 0; i < totflow; ++i) {  
    std::ostringstream leftSubnet, rightSubnet;  
    leftSubnet << "10.2." << i + 1 << ".0";  
    rightSubnet << "10.3." << i + 1 << ".0";  
  
    address.SetBase (Ipv4Address(leftSubnet.str().c_str()), "255.255.255.0");  
    address.Assign (leftLeafDevices[i]);  
  
    address.SetBase (Ipv4Address(rightSubnet.str().c_str()), "255.255.255.0");  
    rightInterfaces.push_back(address.Assign (rightLeafDevices[i]));  
}  
Ipv4GlobalRoutingHelper::PopulateRoutingTables ();  
  
uint16_t sinkPort = 9000;  
for (uint32_t i = 0; i < totflow; ++i) {
```

- I used the OnOffHelper and PacketSinkHelper classes to handle the underlying socket creation. Specifically, the lines PacketSinkHelper sinkHelper ("ns3::TcpSocketFactory", sinkAddress); and OnOffHelper clientHelper ("ns3::TcpSocketFactory", sinkAddress); dictate the socket type.
- By passing "ns3::TcpSocketFactory" as the first argument to these helpers, I instructed the ns-3 applications to automatically instantiate, bind, and connect TCP sockets for my traffic generation and reception.

3.) How did I evaluate the fairness of the network flows?

```
FlowMonitorHelper flowmonHelper;
Ptr<FlowMonitor> flowmon = flowmonHelper.InstallAll ();

Simulator::Stop (Seconds (10.0));
Simulator::Run ();

flowmon->CheckForLostPackets ();
Ptr<Ipv4FlowClassifier> classifier = DynamicCast<Ipv4FlowClassifier> (flowmonHelper.
std::map<FlowId, FlowMonitor::FlowStats> stats = flowmon->GetFlowStats ();

double sthroughput = 0, sthroughputSq = 0;
int validFlows = 0;

for (auto const &i : stats) {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (i.first);
    std::ostringstream destStream; destStream << t.destinationAddress;

    if (destStream.str().find("10.3.") == 0 && t.destinationPort == sinkPort) {
        double throughput = i.second.rxBytes * 8.0 / (10.0 - 2.0) / 1000000.0;
        if (throughput > 0) {
            sthroughput += throughput;
            sthroughputSq += (throughput * throughput);
            validFlows++;
        }
    }
}

if (validFlows > 0 && sthroughputSq > 0) {
    double jainsIndex = (sthroughput * sthroughput) / (validFlows * sthroughputSq);
    std::cout << jainsIndex << std::endl;
} else {
    std::cout << 0.0000 << std::endl;
}
```

- I used the FlowMonitorHelper to automatically track the statistics of all packets traversing my network. Once the simulation concluded, I looped through the FlowStats collected by the monitor.
- I filtered out only the valid data flows bound for my specific sink port and calculated their throughput in Megabits per second. Finally, I used these individual throughput values to compute Jain's Fairness Index at the very end of my script

4.)How did I implement the custom logic for TcpAshish?

- I built the TcpAshish class as a child of ns3::TcpNewReno, which meant I kept all standard TCP behaviors by default. To implement my custom logic, I overrode the PktsAacked function to continuously track the minimum and current Round Trip Time (RTT).
- I then overrode the IncreaseWindow function to alter how the algorithm behaves during Slow Start. Instead of blindly growing the window until a packet drops, I added logic that checks if the current RTT spikes 4 milliseconds above the minimum RTT. If it does, I forcefully set the slow start threshold (ssthresh) to the current window size, kicking the TCP flow out of Slow Start early to prevent congestion.

5.)How was the congestion window traced?

```
private:
    Time m_delayMin;
    Time m_currRtt;
    bool m_thresholdFound;
};

NS_OBJECT_ENSURE_REGISTERED (TcpAshish);

static void CwndChange (Ptr<OutputStreamWrapper> stream, uint32_t oldCwnd, uint32_t newCwnd) {
    *stream->GetStream () << Simulator::Now ().GetSeconds () << "\t" << oldCwnd << "\t" << newCwnd << std::endl;
}

static void RttChange (Ptr<OutputStreamWrapper> stream, Time oldRtt, Time newRtt) {
    *stream->GetStream () << Simulator::Now ().GetSeconds () << "\t" << oldRtt.GetSeconds () << "\t" << newRtt.GetSeconds () << std::endl;
}

static void SsthreshChange (Ptr<OutputStreamWrapper> stream, uint32_t oldSsthresh, uint32_t newSsthresh) {
    *stream->GetStream () << Simulator::Now ().GetSeconds () << "\t" << oldSsthresh << "\t" << newSsthresh << std::endl;
}
```

- I used the Config::ConnectWithoutContextFailSafe() function to hook into the ns-3 trace sources. I provided it with the explicit path to the first socket on a specific node (e.g., /NodeList/[NodeId]/\$ns3::TcpL4Protocol/SocketList/0/CongestionWindow). I then linked this trace source to my custom CwndChange function using MakeBoundCallback.
- Whenever the cwnd value changed during the simulation, it triggered my callback, which then wrote the exact timestamp.

Question 1 and Question 2 :

```
class TcpAshish : public TcpNewReno {
public:
    static TypeId GetTypeId (void) {
        static TypeId tid = TypeId ("ns3::TcpAshish")
            .SetParent<TcpNewReno> ()
            .SetGroupName ("Internet")
            .AddConstructor<TcpAshish> ();
        return tid;
    }

    TcpAshish () : TcpNewReno (), m_delayMin (Time::Max ()), m_thresholdFound (false) {}

    TcpAshish (const TcpAshish& sock)
        : TcpNewReno (sock),
          m_delayMin (sock.m_delayMin),
          m_currRtt (sock.m_currRtt),
          m_thresholdFound (sock.m_thresholdFound) {}

    virtual ~TcpAshish () {}

    virtual std::string GetName () const override { return "TcpAshish"; }

    virtual Ptr<TcpCongestionOps> Fork () override {
        return CreateObject<TcpAshish> (*this);
    }

    virtual void PktsAacked (Ptr<TcpSocketState> tcb, uint32_t segmentsAacked, const Time& rtt) override {
        if (rtt.IsStrictlyPositive ()) {
            if (m_delayMin == Time::Max () || rtt < m_delayMin) {
                m_delayMin = rtt;
            }
            m_currRtt = rtt;
        }
        TcpNewReno::PktsAacked(tcb, segmentsAacked, rtt);
    }

    virtual void IncreaseWindow (Ptr<TcpSocketState> tcb, uint32_t segmentsAacked) override {
        if (tcb->m_cWnd < tcb->m_ssThresh) {
            if (!m_thresholdFound && m_delayMin != Time::Max ()) {
                Time threshold = Milliseconds(4);
                if (m_currRtt > m_delayMin + threshold) {
                    m_thresholdFound = true;
                    tcb->m_ssThresh = tcb->m_cWnd;
                }
            }

            if (!m_thresholdFound) {
                tcb->m_cWnd += tcb->m_segmentSize;
            }
        } else {
            TcpNewReno::IncreaseWindow(tcb, segmentsAacked);
        }
    }

private:
    Time m_delayMin;
    Time m_currRtt;
    bool m_thresholdFound;
};
```

Class Inheritance :

- The TcpAshish class is built directly on top of the standard TcpNewReno congestion control algorithm.
- By inheriting from NewReno, the code keeps proven mechanics for things like Congestion Avoidance. It only overrides specific functions like PktsAacked and IncreaseWindow to change how the initial speed-up phase work

Tracking the Network Baseline (PktsAacked Function) :

- Custom tracking for m_delayMin (the absolute fastest round-trip time) and m_currRtt (the most recent round-trip time).
- Every time a packet is acknowledged, the code captures the RTT. If this new RTT is strictly positive and lower than any previously recorded time, it saves it as m_delayMin to serve as a baseline of an empty network.

Intercepting Slow Start (IncreaseWindow Function):

- The custom logic only kicks in when the congestion window (cWnd) is smaller than the slow start threshold (ssThresh). This is the aggressive Slow Start phase.
- Instead of blindly doubling the sending rate until a packet drops like standard TCP, this code actively monitors the network delay to spot congestion forming in router queues.

The 4 ms Threshold :

- The code defines a strict warning buffer of 4 milliseconds added to your baseline m_delayMin.
- During the Slow Start phase, it constantly checks if the current RTT (m_currRtt) has grown larger than that m_delayMin + 4ms limit. If it has, it means packets are starting to wait in line at network routers.

Preemptive Speed Control:

- The moment that 4ms delay threshold is crossed, the code sets a flag called m_thresholdFound to true.
- It immediately stops the aggressive Slow Start phase. It does this by setting the slow start threshold (ssThresh) equal to the current window size (cWnd), capping the rapid growth before any packets actually drop.

Returning to Standard Behavior:

- This custom delay-sensing trick is only meant for the very beginning of the connection.
- If the threshold has already been found or if the connection has naturally moved into Congestion Avoidance, the code simply calls `TcpNewReno::IncreaseWindow`. This hands control back to standard TCP rules for safe and steady data transfer.

Question3 :

1.)The Dumbbell Topology & 2Mbps Bottleneck

```
PointToPointHelper p2pleaf, p2pbotn;  
p2pleaf.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
p2pleaf.SetChannelAttribute ("Delay", StringValue ("2ms"));  
  
p2pbotn.SetDeviceAttribute ("DataRate", StringValue ("2Mbps"));  
p2pbotn.SetChannelAttribute ("Delay", StringValue ("2ms"));
```

Explanation :

Classic network simulation setup that physically resembles a dumbbell. It consists of multiple end-host nodes (the weights) on both sides that connect to two central routers. Data enters the central router from the leaves at a potential rate of 5Mbps, but the router can only forward that data across the inner link at 2Mbps. The router acts as a funnel.

2.)Flow Count

```
int main (int argc, char * argv[]) {  
    uint32_t totflow = 4;  
    bool mix = false;  
  
    CommandLine cmd;  
    cmd.AddValue ("nFlows", "Total number of TCP flows", totflow);  
    cmd.AddValue ("isMixed", "If true, half flows use TcpAshish", mix);  
    cmd.Parse (argc, argv);  
  
    NodeContainer leftNodes, rightNodes, routers;  
    leftNodes.Create (totflow);  
    rightNodes.Create (totflow);  
    routers.Create (2);  
  
    PointToPointHelper p2pleaf, p2pbotn;  
    p2pleaf.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));  
    p2pleaf.SetChannelAttribute ("Delay", StringValue ("2ms"));
```

Explanation :

We took the totalflow , using TcpAshish or not via cmdline argument instead of hardcoding and changing the value for each run.

3.)Metric :

```
if (mix) {  
    Config::ConnectWithoutContextFailSafe ("/NodeList/" + std::to_string(nodeCustomId) + "/" + $ns3::TcpL4Protocol/SocketList/0/CongestionWindow",  
        MakeBoundCallback (&CwndChange, ascii.CreateFileStream (prefix + "_TcpAshish_cwnd.txt")));  
    Config::ConnectWithoutContextFailSafe ("/NodeList/" + std::to_string(nodeCustomId) + "/" + $ns3::TcpL4Protocol/SocketList/0/SlowStartThreshold",  
        MakeBoundCallback (&SsthreshChange, ascii.CreateFileStream (prefix + "_TcpAshish_ssthresh.txt")));  
    Config::ConnectWithoutContextFailSafe ("/NodeList/" + std::to_string(nodeCustomId) + "/" + $ns3::TcpL4Protocol/SocketList/0/RTT",  
        MakeBoundCallback (&RttChange, ascii.CreateFileStream (prefix + "_TcpAshish_rtt.txt")));  
}
```

Explanation :

This event driven code works into the ns3 core to continuously monitor the TCP socket's internal state. Each update instantly captures the simulation timestamp and new metric value, logging them directly into sequential .txt files. This creates a time-series record of the network behavior, providing the datasets needed to visualize the algorithm's performance and queue reactions later using Gnuplot.

4.)Network Routing

```
Ipv4GlobalRoutingHelper::PopulateRoutingTables ();  
  
uint16_t sinkPort = 9000;  
for (uint32_t i = 0; i < totflow; ++i) {  
    Address sinkAddress (InetSocketAddress (rightInterfaces[i].GetAddress (0), sinkPort));  
    PacketSinkHelper sinkHelper ("ns3::TcpSocketFactory", sinkAddress);  
    ApplicationContainer sinkApp = sinkHelper.Install (rightNodes.Get (i));  
    sinkApp.Start (Seconds (1.0));  
    sinkApp.Stop (Seconds (10.0));  
  
    OnOffHelper clientHelper ("ns3::TcpSocketFactory", sinkAddress);  
    clientHelper.SetAttribute ("OnTime", StringValue ("ns3::ConstantRandomVariable[Constant=1]"));  
    clientHelper.SetAttribute ("OffTime", StringValue ("ns3::ConstantRandomVariable[Constant=0]"));  
    clientHelper.SetAttribute ("DataRate", DataRateValue (DataRate ("5Mbps")));  
    clientHelper.SetAttribute ("PacketSize", UIntegerValue (1024));  
  
    ApplicationContainer clientApp = clientHelper.Install (leftNodes.Get (i));  
    clientApp.Start (Seconds (2.0));  
    clientApp.Stop (Seconds (10.0));  
}
```

Explanation :

This code snippet configures the network routing and generates the actual data traffic for the simulation. It first populates the global routing tables so all nodes know how to communicate across the topology. Then, it uses a loop to create multiple independent traffic flows: it installs a receiver

application (PacketSink) on the right-side nodes that starts listening at the 1.0-second mark, and a sender application (OnOffApplication) on the left-side nodes. These senders are configured to transmit a continuous, non-stop 5Mbps stream of 1024-byte TCP packets starting at 2.0 seconds, with all data transfer terminating at 10.0 seconds.

4.)Final Throughput

```
for (auto const &i : stats) {
    Ipv4FlowClassifier::FiveTuple t = classifier->FindFlow (i.first);
    std::ostringstream destStream; destStream << t.destinationAddress;

    if (destStream.str().find("10.3.") == 0 && t.destinationPort == sinkPort) {
        double throughput = i.second.rxBytes * 8.0 / (10.0 - 2.0) / 1000000.0;
        if (throughput > 0) {
            sthroughput += throughput;
            sthroughputSq += (throughput * throughput);
            validFlows++;
        }
    }

    if (validFlows > 0 && sthroughputSq > 0) {
        double jainsIndex = (sthroughput * sthroughput) / (validFlows * sthroughputSq);
        std::cout << jainsIndex << std::endl;
    } else {
        std::cout << 0.0000 << std::endl;
    }
}
```

Explanation :

This code analyzes network traffic statistics to measure how fairly bandwidth is distributed among different connections. It filters for specific data flows heading to a targeted '10.3' subnet and port, calculating their individual throughput in Megabits per second over an 8 second simulation window. By accumulating the sum and the squared sum of these throughputs, the script computes and prints Jain's Fairness Index. This mathematical metric evaluates the overall network balance to determine if all active connections are receiving an equal share of the available bandwidth or if some streams are dominating the channel.

Question 4:

Shell Script for running and storing the network log in .txt file

```
$ as8_run.sh
1  #!/bin/bash
2
3  mkdir -p as8_results
4  output_file="as8_results/jains_index_table.md"
5
6  echo "running the codes "
7  ./ns3 build
8
9  {
10 |     echo "| Flows          | 4      | 8      | 16      | 20      |"
11 } | tee "$output_file"
12
13 printf "| NewReno (Baseline)  |" | tee -a "$output_file"
14 for n in 4 8 16 20; do
15     res=$(./ns3 run "as8_que1 --nFlows=$n --isMixed=false" 2>/dev/null | tail -n 1)
16     printf "%-6.4f |" "$res" | tee -a "$output_file"
17 done
18 echo "" | tee -a "$output_file"
19
20 printf "| NewReno & TcpAshish |" | tee -a "$output_file"
21 for n in 4 8 16 20; do
22     res=$(./ns3 run "as8_que1 --nFlows=$n --isMixed=true" 2>/dev/null | tail -n 1)
23     printf "%-6.4f |" "$res" | tee -a "$output_file"
24 done
25 echo "" | tee -a "$output_file"
26
27 mv Baseline_*.txt as8_results/ 2>/dev/null
28 mv Mixed_*.txt as8_results/ 2>/dev/null
29 mv *.pcap as8_results/ 2>/dev/null
```

COMMAND USED : `chmod +x as8_run.sh`
`./as8_run.sh`

Output :

This bash script automates the execution and data collection for your ns-3 network simulations. It compiles the simulator and sequentially runs two distinct test scenarios a pure NewReno baseline and a mixed NewReno/TcpAshish environment scaling the network traffic across 4, 8, 16, and 20 concurrent flows. For each run, it extracts the final output metric (Jain's Fairness Index) and dynamically formats it into a comparative Markdown table. After completing the tests, it acts as a cleanup utility by moving all generated time-series.txt logs like [Baseline/Mixed]_[Flows]_[CCA]_[cwnd/rtt] , into a dedicated `as8_results` directory for organized post-analysis.

Output :

```
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ chmod +x as8_run.sh
ashish@ashish-ASUS-TUF-Gaming-F15-FX507VV-FX507VV:~/ns-allinone-3.44/ns-3.44$ ./as8_run.sh
running the codes
[ 1%] Building CXX object scratch/CMakeFiles/scratch_as8_que1.dir/as8_que1.cc.o
[ 1%] Linking CXX executable ../../build/scratch/ns3.44-as8_que1-default
Finished executing the following commands:
/usr/bin/cmake --build /home/ashish/ns-allinone-3.44/ns-3.44/cmake-cache -j 15
| Flows          | 4      | 8      | 16     | 20     |
| NewReno (Baseline) | 0.9852 | 0.9888 | 0.9932 | 0.9943 |
| NewReno & TcpAshish | 0.9999 | 0.9887 | 0.9915 | 0.9966 |
```

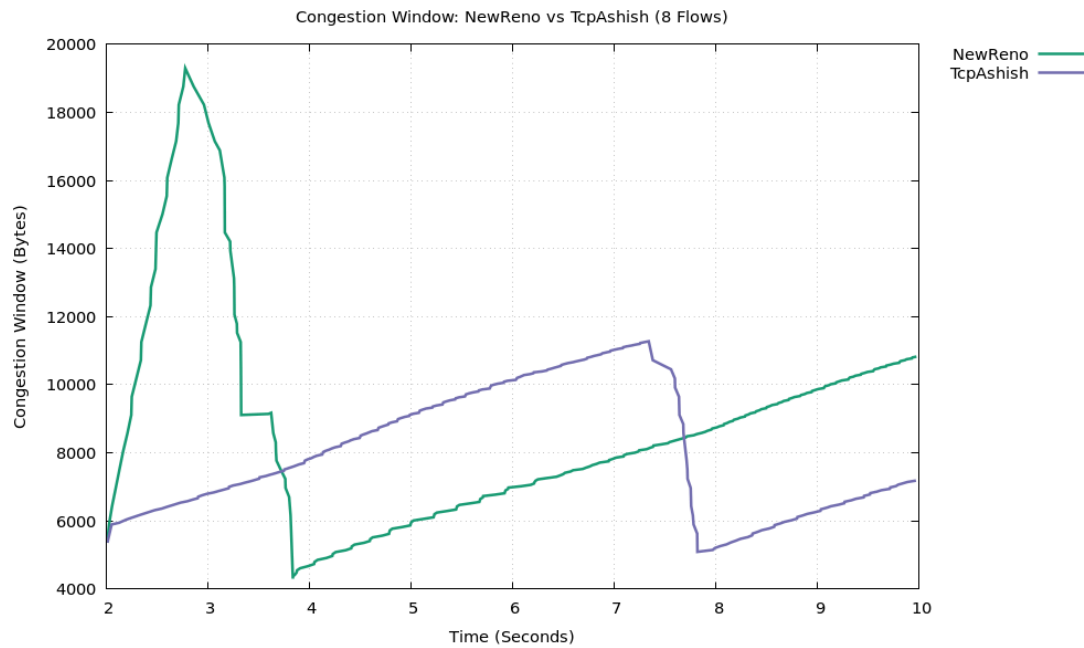
Table :

Flows	4	8	16	20
NewReno (Baseline)	0.9852	0.9888	0.9932	0.9943
NewReno & TcpAshish	0.9999	0.9887	0.9915	0.9966

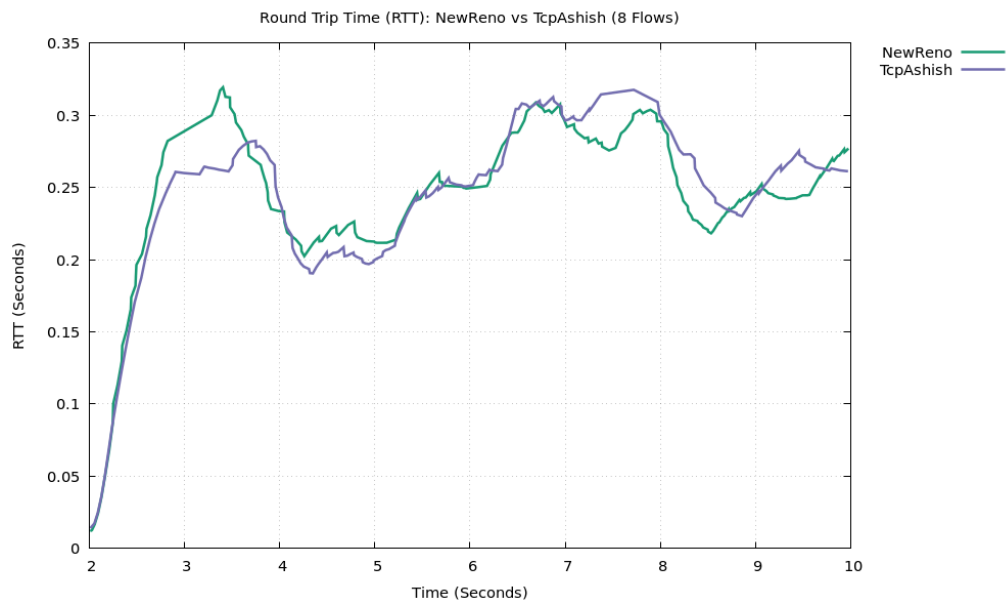
Question 5 :

COMMAND USED : `gnuplot as8_gnuplot.p`

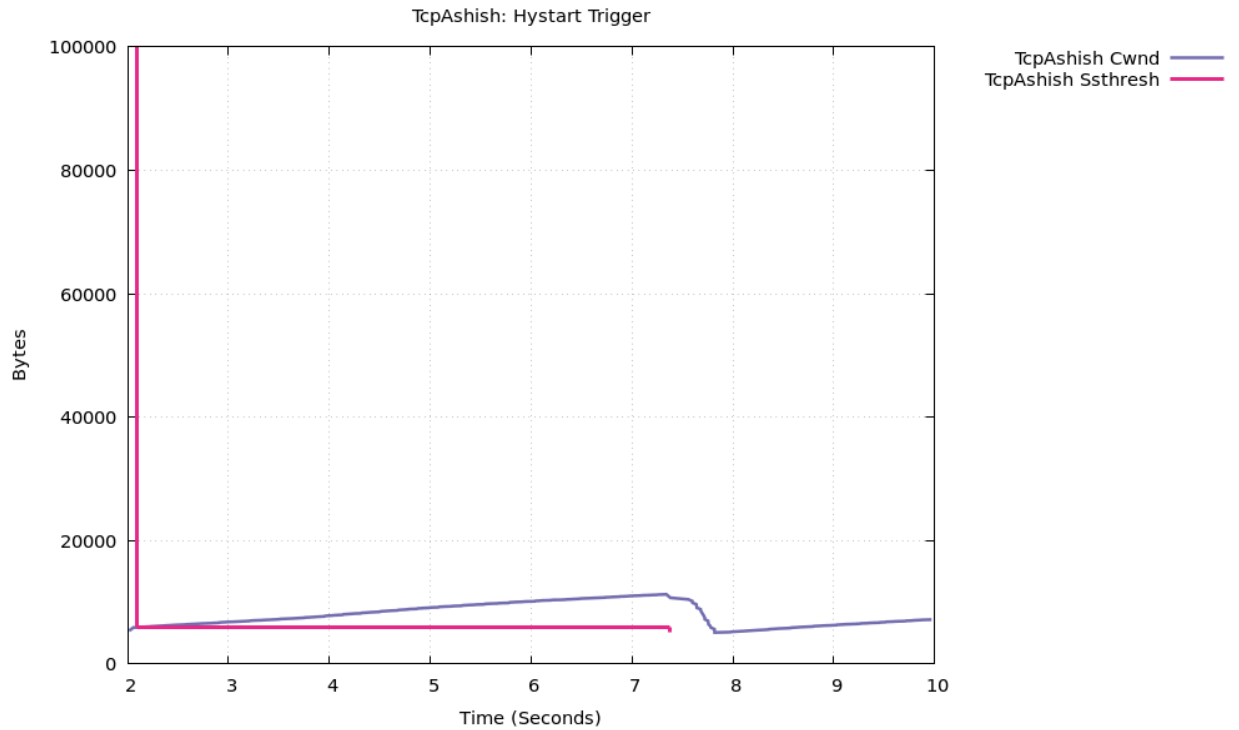
1.) Congestion Window (cwnd_plt.png)



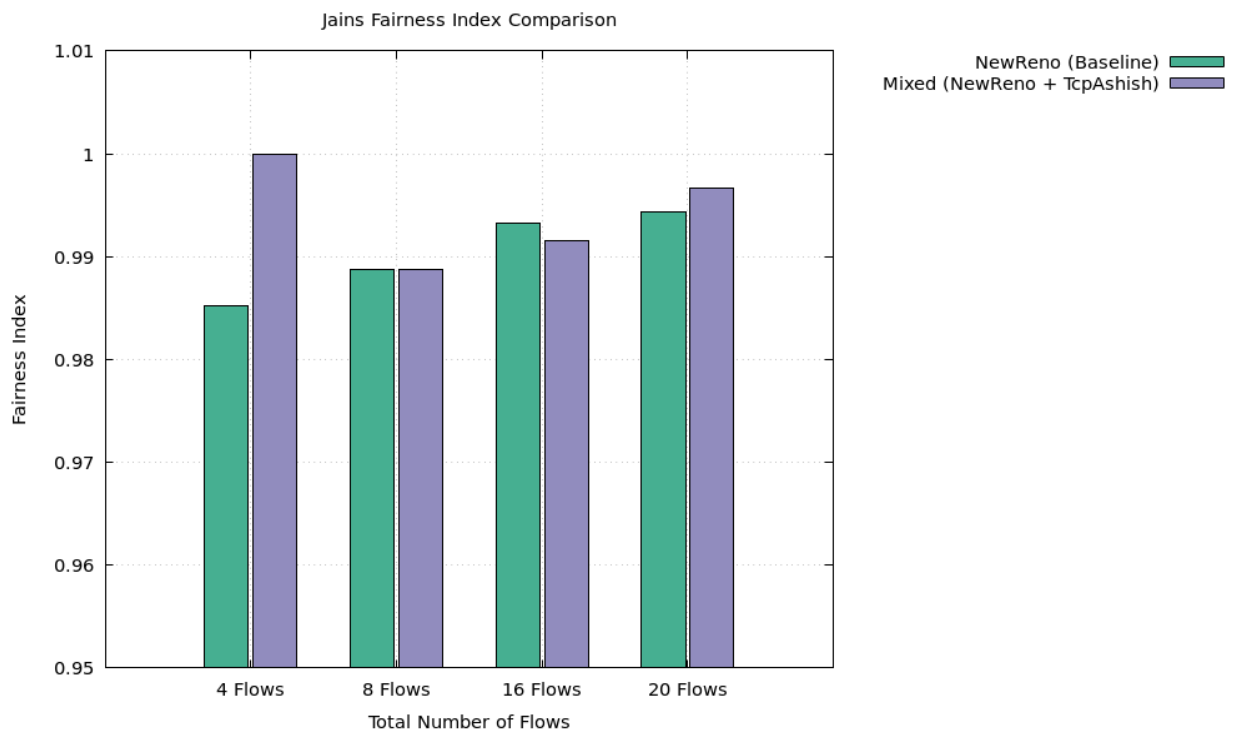
2.) Round Trip Time (RTT) (rtt_plt.png)



3.) Ssthresh (ssthresh_plt.png)



4.) Fairness Index Graph (through_plt.png)



Question 6

1.) Congestion Window Plot

Observations

- The standard NewReno protocol shows a massive spike in its congestion window after starting, peaking near 19000 bytes before suffering heavy collapse.
- The custom TcpAshish protocol completely lacks this initial spike, instead showing a much slower, steady linear growth starting from roughly 6,000 bytes.
- Both protocols experience a significant drop in their window size at different times (NewReno around 3 seconds and TcpAshish around 7 seconds), followed by a slow, parallel recovery slope.

Inferences

- Standard NewReno blindly doubles its sending rate during its Slow Start phase until it completely overwhelms the bottleneck router, resulting in massive packet loss and the severe window crashes seen on the graph.
- TcpAshish successfully utilizes its delay-sensing mechanism to detect network queue, preemptively halting the aggressive Slow Start phase before packet loss actually occurs.
- By actively controlling the initial burst of traffic, TcpAshish prevents the early catastrophic buffer overflows that NewReno causes leading to a stable data transmission flow over the network.

2.) Round Trip Time Plot

Observations

- **Initial Delay Spike:** Between the 2 second and 3.5 second, NewReno experiences a high peak in Round Trip Time (reaching roughly 0.32 seconds). In contrast, TcpAshish flattens out its RTT growth earlier and at a significantly lower peak.
- **Cyclical RTT Behavior:** After the initial startup phase both protocols display continuous wave patterns. The RTT repeatedly climbs toward 0.3 seconds and then drops back down to the 0.2 second range.
- **Long-Term Alignment:** In the later stages of the simulation (from 5 seconds onwards), the RTT curves for both NewReno and TcpAshish are close to each other sharing similar peaks, and overall trend lines.

Inferences

- **Preemptive Queue Control:** NewReno's aggressive blind data transmission during slow start rapidly fills the bottleneck router's buffers causing the massive initial spike. TcpAshish successfully uses its delay sensing mechanism to detect this queue buildup, capping its transmission rate to prevent that extreme initial congestion.
- **Bottleneck Buffer Dynamics:** The continuous wave pattern indicates that the bottleneck router's queue is constantly filling up (RTT increasing) and then draining (RTT to decreasing). This happens because the TCP flows eventually drop packets, change their speeds to recover and then slowly back up.
- **Shared Congestion Avoidance:** Because the custom TcpAshish logic is designed to hand control back to standard NewReno after the initial threshold is met, its long term delay characteristics naturally converge with the baseline protocol. The delay sensing trick works perfectly to smooth out the turbulent start, but it relies on standard mechanics for long-term network stabilization.

3.) Ssthreshold (Hystart) Plot

Observations

- **Immediate Ssthresh Collapse:** At the very start of the connection (2.0 seconds), the Slow Start Threshold instantly crashes from an extremely high value straight down to roughly 6000 bytes.
- **Linear Cwnd Growth:** Instead of a sharp exponential curve, the Congestion Window begins at that same 6000 byte mark and grows in a slow steady straight line upward until it reaches about 11000 bytes.
- **The 7.3-Second Drop:** Around the 7.3 second mark, the steady growth of the Congestion Window abruptly ends with a drop back down to roughly 5000 bytes, while the Ssthresh line simultaneously disappears or resets.

Inferences

- **The Tripwire Worked Instantly:** That vertical drop in the pink line at 2.0 seconds is the visual proof of your custom code working. The 4 millisecond delay threshold was detected almost immediately, forcing the ssthresh variable to snap down to the current cwnd value to hit the brakes.
- **Bypassing Slow Start:** Because the threshold was lowered instantly to meet the current window size, the connection was forced to skip aggressive Slow Start phase entirely. The purple line being constantly above the pink line confirms the connection operated entirely in the safer, linear "Congestion Avoidance" phase.
- **Eventual Physical Congestion:** The sharp drop at 7.3 seconds shows the linear growth, the bottleneck router eventually filled up and dropped a packet. The standard NewReno recovery mechanics kicked in, cutting the window size to clear the physical network jam.

4.) Fairness Index Plot

Observations

- **Perfect Fairness at Low Scale:** In the 4-flow scenario, the Mixed environment (NewReno + TcpAshish) achieves a perfect Jain's Fairness Index of 1.0, whereas the pure NewReno Baseline drops noticeably lower to approximately 0.985.
- **Convergence at Mid-Scale:** At 8 flows, the fairness scores for both the Baseline and Mixed configurations are virtually identical. At 16 flows, the pure NewReno Baseline actually slightly outperforms the Mixed environment by a tiny margin.
- **High Baseline Across the Board:** Regardless of the flow count or the protocols used, the fairness index never drops below 0.98 in any test. At the highest congestion level (20 flows), both setups remain highly fair, with the Mixed environment slightly out performing the Baseline again.

Inferences

- **Early Domination Prevention:** At lower flow counts (4 flows), standard NewReno's aggressive initial burst allows some connections to grab an unequal share of the bandwidth before dropping packets. TcpAshish's smoother start prevents this, leading to mathematically perfect bandwidth sharing.
- **Standardization Under Heavy Load:** As the network gets heavily congested (8 and 16 flows), the bottleneck router is constantly full. In this state, the initial Slow Start phase matters less, and the network relies entirely on standard TCP Congestion Avoidance mechanics naturally causing their fairness scores to equalize.
- **Network Friendliness:** The most important inference is that TcpAshish is a safe, friendly protocol. Because the Mixed environment maintains extremely high fairness scores alongside pure NewReno flows, it proves the custom algorithm does not aggressively steal bandwidth or starve standard TCP connections out of the network.

Justification for the Graphs :

1. Congestion Window (cwnd) Plot:

The problem with standard NewReno is that it has no idea the network is reaching its limit until a packet actually drops which explains that massive spike and subsequent dive we see in the data. It just keeps doubling its speed until something breaks. TcpAshish avoids that by actually paying attention. It constantly checks for delays; the moment it senses that packets are taking a little too long to get back, it realizes a traffic jam is starting to form. Instead of crashing, it downshifts to a steady, safe pace before a single packet is lost.

2. Round Trip Time (RTT) Plot:

RTT graph is a measure of the queue at the router. **NewReno** tends to cause a massive spike because it charges in so fast and aggressively that it completely fills the queue, forcing every packet to sit in a long line. **TcpAshish** handles things much more gracefully. Because it's listening for delays, it realizes when things are getting crowded reduce the ssthresh before it floods the router queue. By keeping that initial burst under control, it keeps the line moving and the wait times much shorter for everyone.

3. Hystart Plot (Ssthresh vs Cwnd):

The dip in the pink line (**ssthresh**) is the exact it realized the 4ms delay limit was hit. In that split second, the code reduces the ssthreshold. By dropping the threshold down to meet the current window size, it shuts down the aggressive doubling in the slow start phase

4. Fairness Index Plot:

The data shows that TcpAshish works well with others. While it takes a totally different approach for setting up a connection up and running. Once that initial startup phase is finished, it steps back and lets standard TCP rules take the wheel. This means it shares perfectly ensuring everyone on the network gets their fair share of bandwidth just like a standard NewReno setup.